



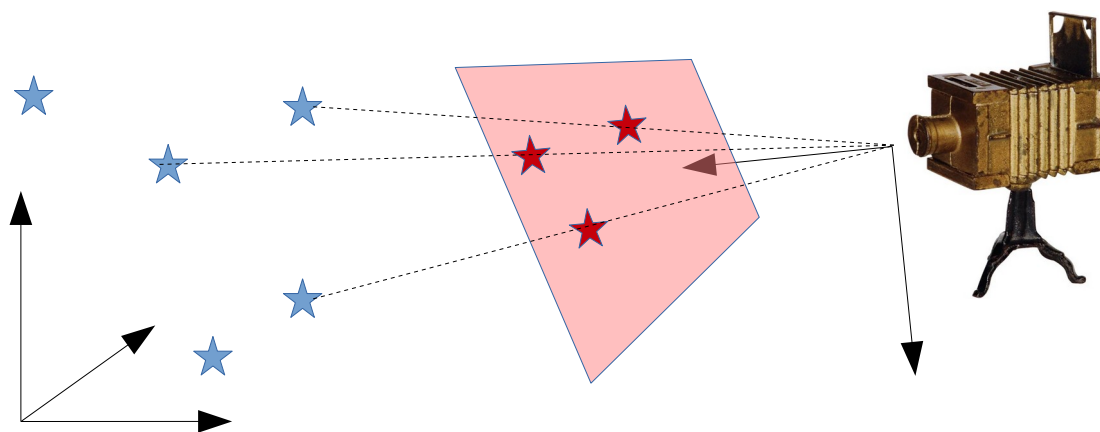
Probabilistic Robotics Course

Projective ICP [Exercise]

Giorgio Grisetti

`grisetti@diag.uniroma1.it`

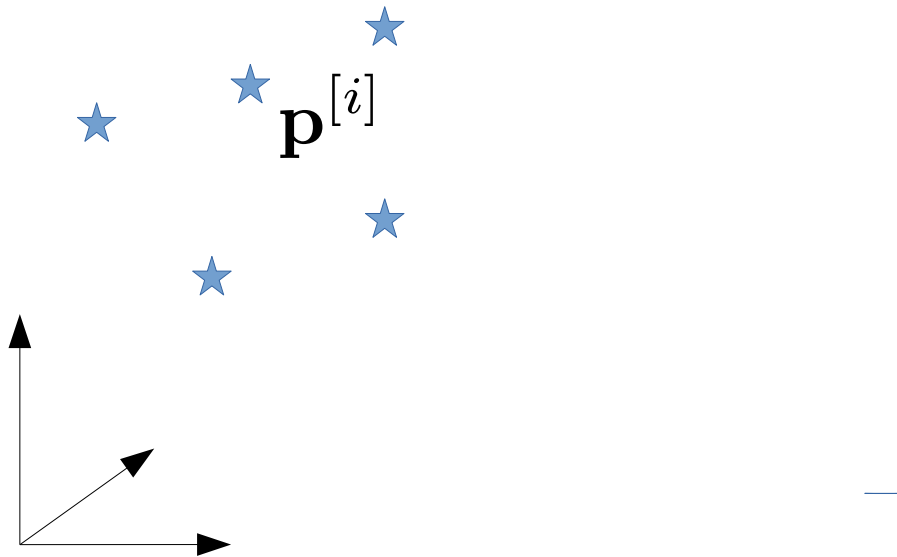
Department of Computer, Control, and Management Engineering
Sapienza University of Rome



Projective Registration

Given

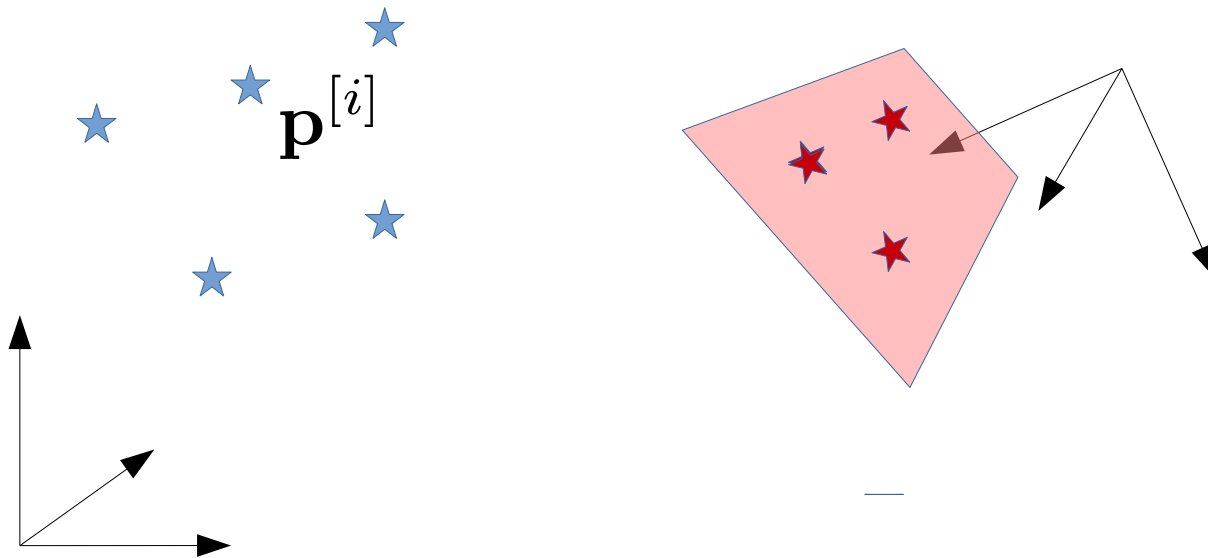
- a set of known 3D points in the world frame



Projective Registration

Given

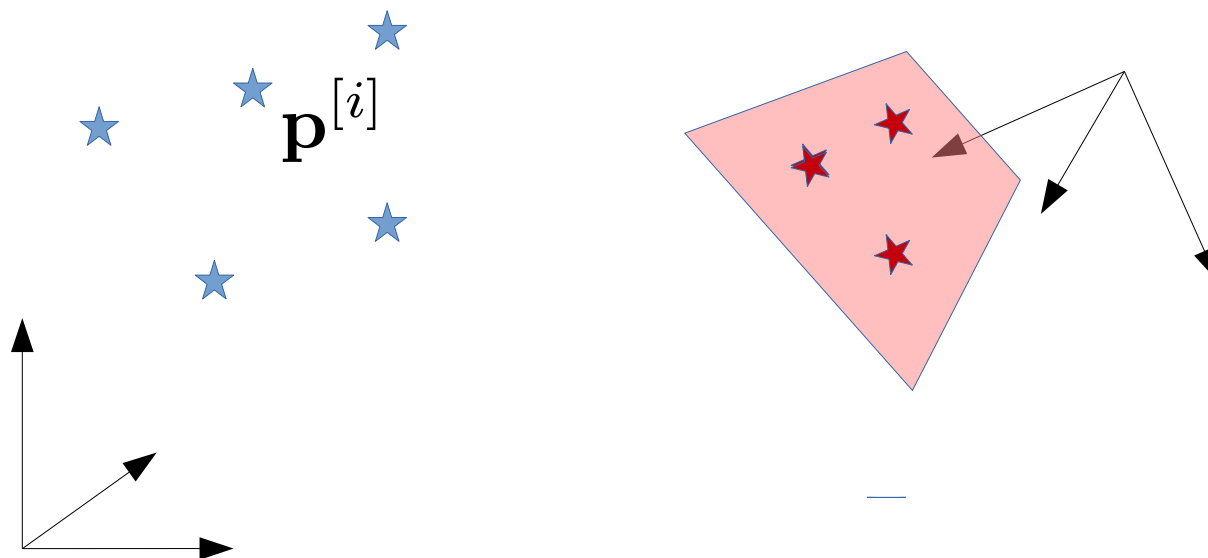
- a set of known 3D points in the world frame
- a set of 2D image projections of these points



Projective Registration

We want to find

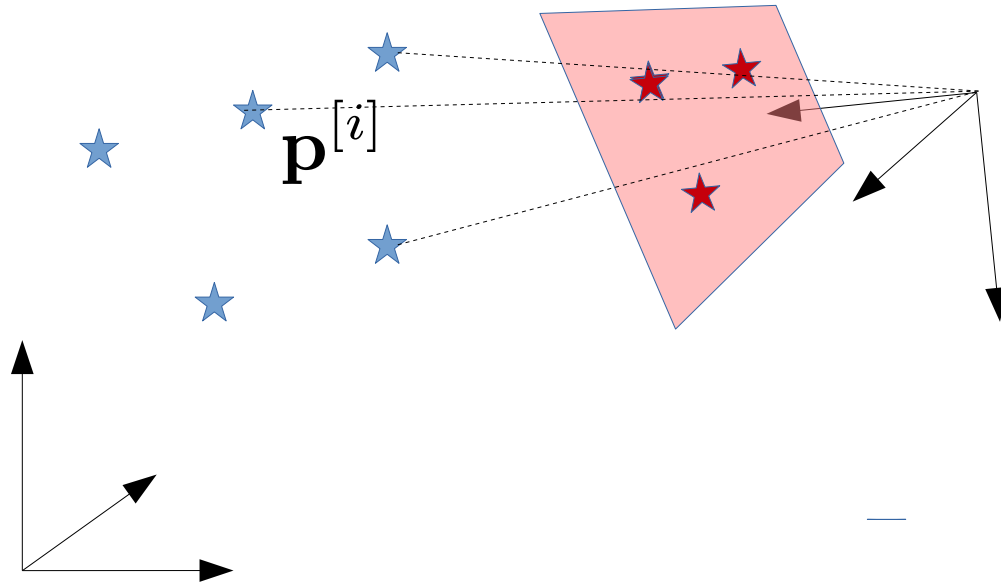
- The relative pose of the observer that minimizes the distance between prediction and measurement on the image plane



Projective Registration

We want to find

- The relative pose of the observer that minimizes the distance between prediction and measurement on the image plane



Data Association is Unknown



Outline

We will walk through a complete programming example that integrates:

- Pinhole camera model
- Data Association
- Least Squares on Manifolds

C++ implementation



How to Proceed?

We will follow these steps

- Construct a Simulation Environment
- Synthesis of the Sensor Model
- Data Association
- Least Squares
- Integrate the parts

After each step, we will do a validation



Simulation Environment

To check the components of the system, it is convenient to create a sandbox environment.

All what we need is a set of randomly drawn 3D points

To render the simulation more realistic, we will put these points along specific patterns in the space (e.g. segments).

This reflects the behavior that we could have for instance running an edge extractor on an image



Simulation Environment

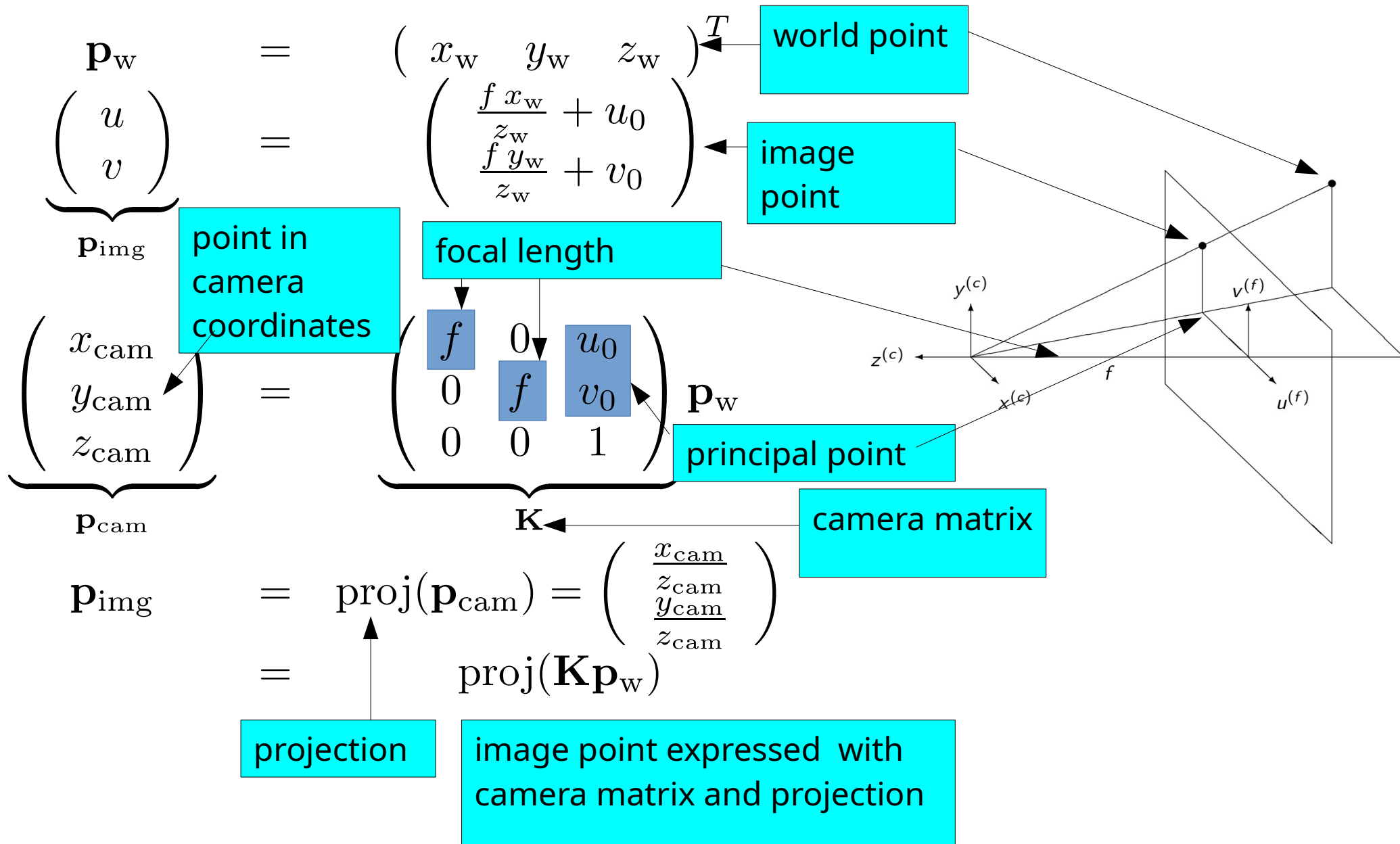
Look at the file `points_utils.{h,cpp}`

There are some utilities to

- generate points in a world
- draw points on an opencv images

**Design your program to be debugged,
through visualization**

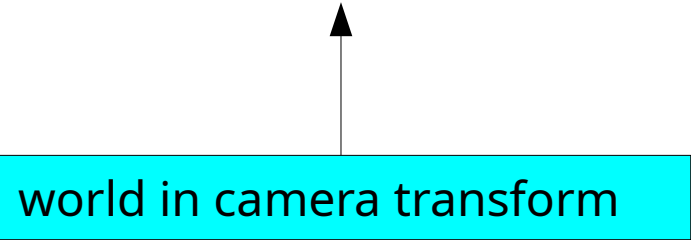
Sensor Model (Pinhole Camera)



Sensor Model

If the reference system of world and camera are not the same, we can compute the projection as follows

$$p_{\text{img}} = \text{proj}(\mathbf{KXp}_w)$$



world in camera transform

The diagram shows a cyan rectangular box containing the text "world in camera transform". A vertical arrow points from the top center of this box to the $\mathbf{X}$ matrix in the projection equation $p_{\text{img}} = \text{proj}(\mathbf{KXp}_w)$ above it.

Implementing the Sensor Model

In a C++ implementation, it makes sense to design a class that will implement the camera functionalities.

- Attributes:
 - pose of the world w.r.t. camera
 - camera matrix
- Functionalities
 - compute image coordinates of a world point, according to the attributes

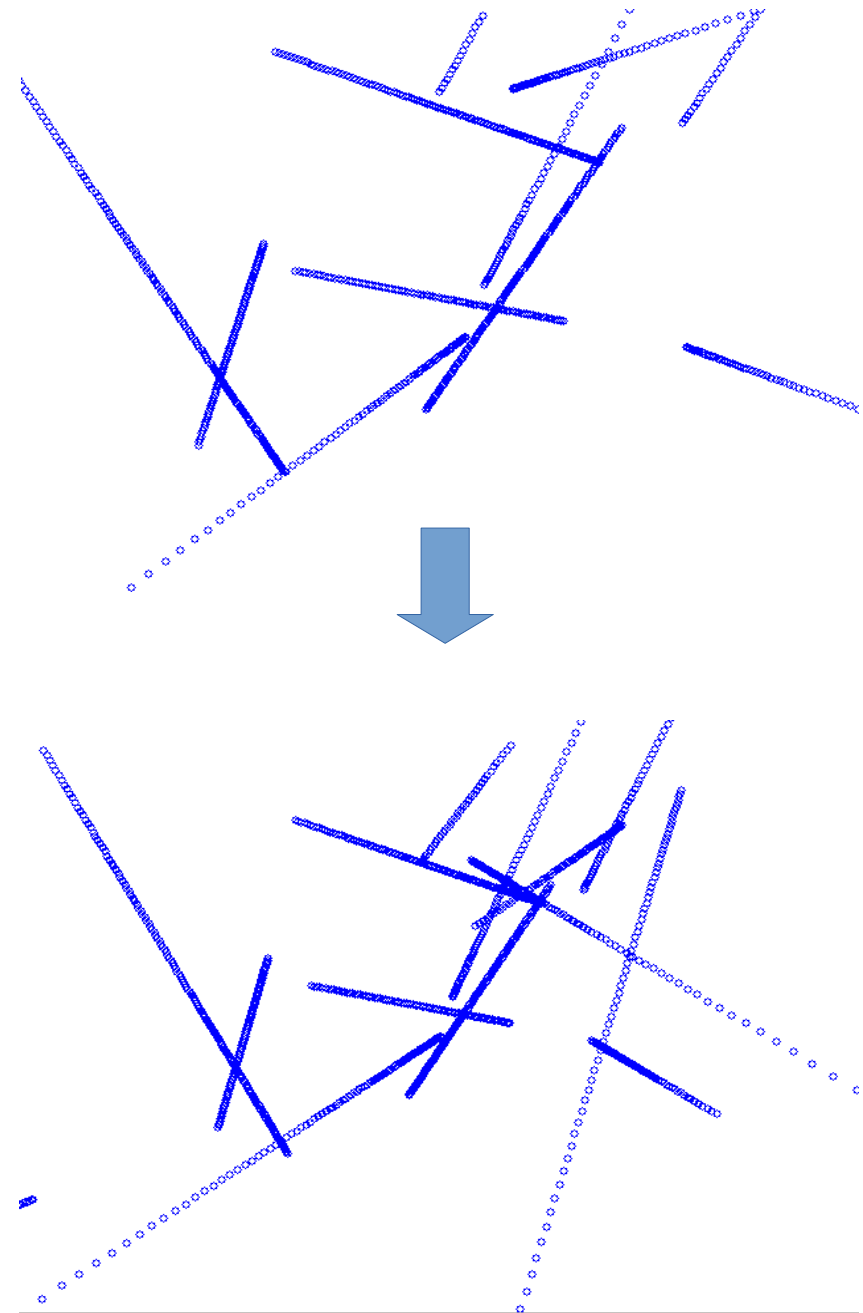
see files `camera.{h,cpp}`

Testing Sensor Model

To check if everything is correct we want to

- construct a simulated environment with a bunch of 3d lines
- place a camera in the world
- move the camera with the arrow keys
- see the image perceived by the camera

Check the file `camera_test.cpp`, and run the corresponding binary





Data Association

If:

- we have a reasonable guess of the camera
- the points are sparse enough

we can approach the data association by a *nearest neighbor strategy*.

Given a transform $\mathbf{X}$ of world w.r.t. camera, we:

- project each point of the world onto the image
- we assign to each of these points the closest measured point on the image

Data Association

During one alignment, we will adjust the association based on our current guess.

A naive implementation requires $O(N^2)$ operations for each least squares iteration.

Considering that

- the measurements on the image do not change during one alignment
- the number of points is potentially large
- the image coordinates lie on a plane

a reasonable strategy is to use a distance map

Data Association

During one alignment, we will

- compute a distance map on the image, based on the measured points (once)
- at each iteration, we will assign to the measured point, the world point whose projection falls closer to it

In a C++ implementation, this is encapsulated in a class `DistanceMapCorrespondenceFinder`

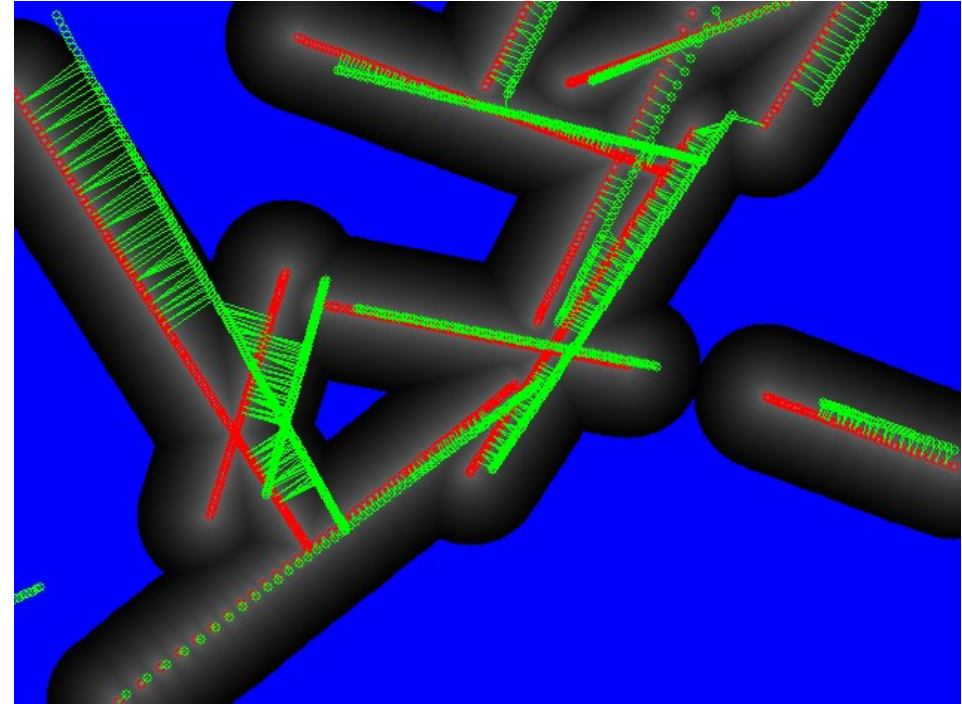
- Attributes:
 - image size
 - image points
 - max_distance
- Functionalities
 - initialization to compute the distance map
 - query for the correspondences

Data Association

To test the data association, we extend our camera_test program and show

- distance map
- computed associations

check out the file
correspondence_finde
r_test.cpp



Least Squares

We will follow the methodology

- State, increments and boxplus

$$\begin{array}{lll} \mathbf{X} \in SE(3) & : & \mathbf{X} = (\mathbf{R}|\mathbf{t}) \\ \Delta \mathbf{x} \in \mathfrak{R}^6 & : & \Delta \mathbf{x} = \underbrace{(x \ y \ z)}_{\mathbf{t}} \underbrace{(\alpha_x \ \alpha_y \ \alpha_z)}_{\alpha}^T \\ \mathbf{X} \boxplus \Delta \mathbf{x} & : & v2t(\Delta \mathbf{x})\mathbf{X} \end{array}$$

- Measurements (Euclidean)

$$\mathbf{z}^{[m]} \in \mathfrak{R}^2 : \mathbf{z}^{[m]} = (u^{[m]} \ v^{[m]})^T$$

Least Squares

- Prediction

$$\mathbf{h}^{[n]}(\mathbf{X}) = \text{proj}(\mathbf{K} \underbrace{\mathbf{X}\mathbf{p}^{[n]}}_{\mathbf{h}_{\text{icp}}^{[n]}(\mathbf{X})})$$

$$\begin{aligned}\hat{\mathbf{p}}^{[n]} &= \mathbf{h}_{\text{icp}}^{[n]}(\mathbf{X}) = \mathbf{X}\mathbf{p}^{[n]} \\ \hat{\mathbf{p}}_{\text{cam}}^{[n]} &= \mathbf{K}\hat{\mathbf{p}}^{[n]}\end{aligned}$$

This is the prediction for 3D ICP we found in the previous lesson

camera coordinates (before homogeneous division)

Least Squares

- Error

$$\begin{aligned} \mathbf{e}^{[n,m]}(\mathbf{X}) &= \mathbf{h}^{[n]}(\mathbf{X}) - \mathbf{z}^{[m]} \\ &= \text{proj}(\mathbf{K} \underbrace{\mathbf{X} \mathbf{p}^{[n]}}_{\mathbf{h}_{\text{icp}}^{[n]}(\mathbf{X})}) - \mathbf{z}^{[m]} \end{aligned}$$

$$\mathbf{e}^{[n,m]}(\mathbf{X} \boxplus \Delta \mathbf{x}) = \text{proj}(\mathbf{K} \mathbf{h}_{\text{icp}}^{[n]}(\mathbf{X} \boxplus \Delta \mathbf{x})) - \mathbf{z}^{[m]}$$

Least Squares

- Jacobian (using chain rule)

$$\begin{aligned}
 \underbrace{\left. \frac{\partial \mathbf{e}^{[n,m]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}} \right|_{\Delta \mathbf{x}=\mathbf{0}}}_{\mathbf{J}^{[n]}} &= \underbrace{\left. \frac{\partial \text{proj}(\mathbf{p})}{\partial \mathbf{p}} \right|_{\mathbf{p}=\hat{\mathbf{p}}_{\text{cam}}^{[n]}}}_{\mathbf{J}_{\text{proj}}(\mathbf{p})} \mathbf{K} \underbrace{\left. \frac{\partial \mathbf{h}_{\text{icp}}^{[n]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}} \right|_{\Delta \mathbf{x}=\mathbf{0}}}_{\mathbf{J}_{\text{icp}}^{[n]}} \\
 &= \mathbf{J}_{\text{proj}} \left(\hat{\mathbf{p}}_{\text{cam}}^{[n]} \right) \mathbf{K} \mathbf{J}_{\text{icp}}^{[n]} \\
 &= \mathbf{J}_{\text{icp}}^{[n]} \\
 &= \mathbf{J}_{\text{proj}}(\mathbf{p}) \begin{pmatrix} \mathbf{I}_{3 \times 3} \mid \lfloor -\hat{\mathbf{p}}^{[n]} \rfloor \times \end{pmatrix} \quad \text{☰} \\
 &= \begin{pmatrix} \frac{1}{z} & 0 & -\frac{x}{z^2} \\ 0 & \frac{1}{z} & -\frac{y}{z^2} \end{pmatrix}
 \end{aligned}$$

Least Squares

In C++ we will write a class that implements the least squares solver

Attributes

- world_points
- image_points
- current camera pose
- kernel_threshold

Functionalities

- initialization
- one iteration(given the correspondences)

Check out the files `picp_solver.*` for more details

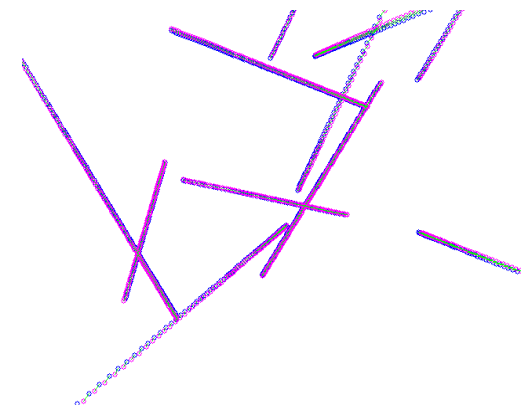
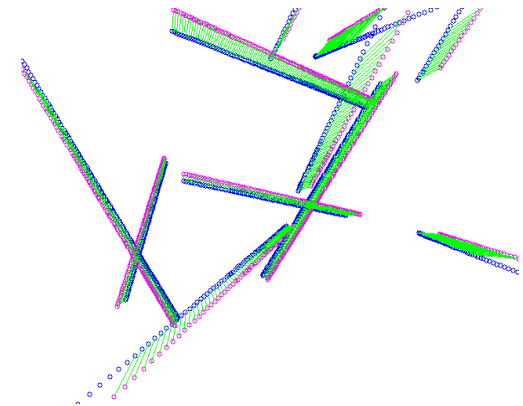
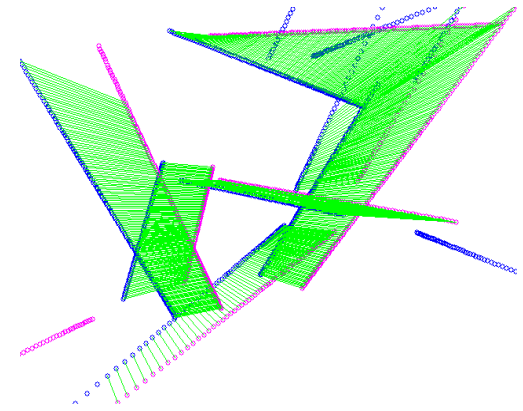
Least Squares

To test our least squares solution, we need to define a problem (with perfect associations) and let the algorithm run

We will modify the camera_test, to get picp_solver_test

arrow keys; move the camera

spacebar: performs one least squares iteration



Least Squares (Hints)

When implementing the least squares part

- validate first the error function (if the system is at the optimum, the error should be 0)
- compute first the numeric jacobians (or use autodiff)
- only if needed (by speed) compute the analytic jacobians

Putting together the parts

Once we have validated each single aspect we proceed to the integration

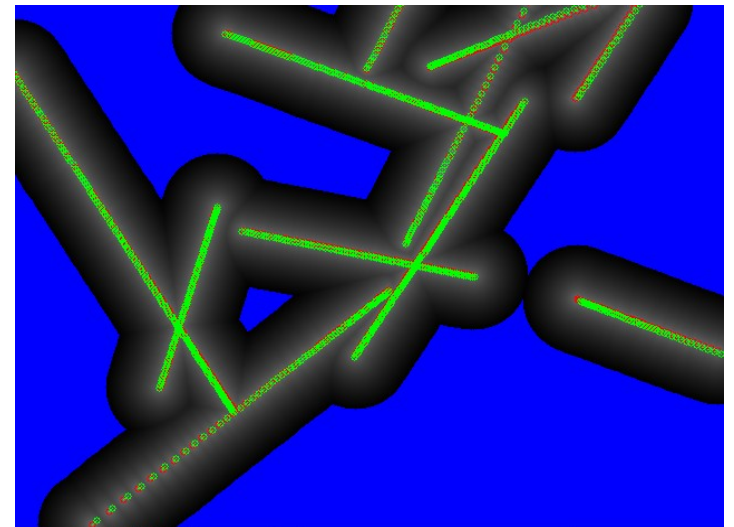
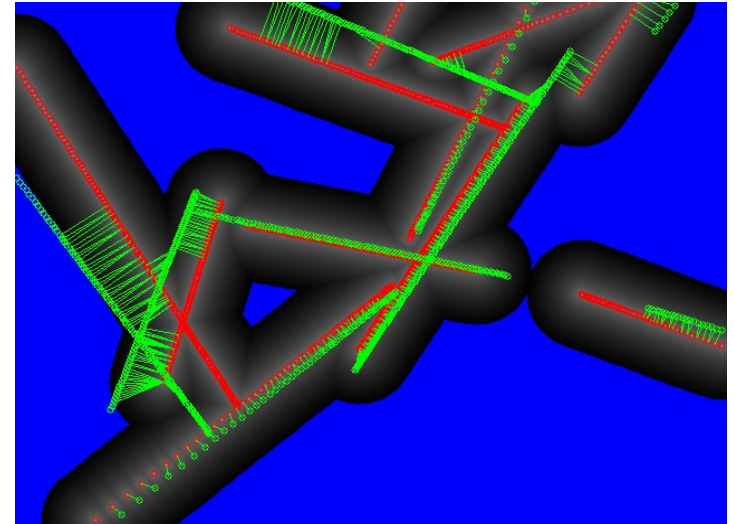
See file

“picp_complete_test.cpp”
and the corresponding
binary

arrow keys; move the camera

spacebar: performs one
association+least squares

round





Conclusions

- Divide a non-trivial problem in smaller tasks that you can solve easily
- Sandbox the individual components of your system in order to validate the behaviors (e.g. by using a simulator) 